



UNIVERSITÀ
di **VERONA**

Dipartimento di Informatica

**Q-Learning in ambienti a ricompensa
sparsa: analisi sperimentale
nell'ambiente DoorKey**

A.A. 2025/2026

Antongiulio Donno VR502818

Indice

1	REINFORCEMENT LEARNING	1
1.1	Introduzione al Reinforcement Learning	1
1.1.1	Conoscenza del Modello	1
1.2	Q-Learning	2
1.2.1	Sample-based Q-Learning	2
1.3	Esplorazione vs Sfruttamento	3
1.4	Spiegazione dei parametri	4
2	AMBIENTE DOORKEY	4
2.1	Ambiente Doorkey	4
2.1.1	Spazio delle Azioni	5
2.2	Scelta dell'algoritmo	6
2.3	Setup dell'Addestramento	6
2.4	Risultati Sperimentali	6
2.5	Analisi e Confronto delle Configurazioni	9

1 REINFORCEMENT LEARNING

1.1 Introduzione al Reinforcement Learning

Il Reinforcement Learning (RL) è un paradigma dell'apprendimento automatico in cui un agente intelligente impara a prendere decisioni ottimali interagendo con un ambiente, con l'obiettivo di massimizzare un segnale di ricompensa cumulativo nel tempo. I concetti fondamentali del RL includono:

- **Trial and error (Apprendimento per prove ed errori):** l'agente esplora l'ambiente in modo attivo, deducendo le strategie ottimali dalle conseguenze delle proprie azioni.
- **Delayed reward (Ricompensa ritardata):** le azioni intraprese possono avere conseguenze e generare ricompense solo a lungo termine.

L'obiettivo formale del RL è la stima accurata della **funzione di valore** $V(s)$ e l'individuazione della **policy ottima** $\pi^*(s)$, ovvero la mappatura stato-azione che massimizza il valore atteso della ricompensa futura.

Il problema è formalizzabile come un Processo Decisionale di Markov (MDP) in cui, a differenza della pianificazione classica:

- La funzione di ricompensa $R(s, a, s')$ è incognita.
- Le probabilità di transizione tra stati $T(s, a, s')$ (la dinamica dell'ambiente) sono incognite.

Di conseguenza, l'agente deve acquisire campioni empirici provando azioni e raccogliendo le relative ricompense.

1.1.1 Conoscenza del Modello

Le strategie per risolvere problemi di RL si dividono in due categorie principali:

- **Metodi Model-based:** tentano di stimare esplicitamente il modello dell'ambiente (le matrici di transizione e di ricompensa). Sfruttando queste stime, l'agente può pianificare le azioni riducendo l'interazione con stati a ricompensa negativa. Risultano più efficienti in termini di campionamento (sample efficiency), ma computazionalmente più complessi.

- **Metodi Model-free:** evitano di costruire un modello dell'ambiente, concentrandosi sull'apprendimento diretto della funzione valore-azione (Q-function) e della policy ottima tramite l'esperienza. Risultano più semplici da implementare e presentano un bias inferiore, sebbene richiedano una maggiore quantità di dati per convergere.

1.2 Q-Learning

Il Q-Learning è il principale algoritmo model-free per il controllo ottimo. Permette di stimare direttamente i valori di $Q(s, a)$ approssimando l'equazione di ottimalità di Bellman senza necessitare della dinamica dell'ambiente:

$$Q_{k+1}(s, a) = \sum_{s'} T(s, a, s') \left(R(s, a, s') + \gamma \max_{a'} Q_k(s', a') \right)$$

1.2.1 Sample-based Q-Learning

Acquisendo una tupla di esperienza (s, a, r, s') , l'algoritmo aggiorna iterativamente la stima della funzione Q fondendo il valore storico con il nuovo target temporale (TD Target):

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

Proprietà e Convergenza Il Q-Learning garantisce la convergenza asintotica alla reale funzione $Q^*(s, a)$ (e quindi alla policy ottima) sotto due condizioni fondamentali:

- **Esplorazione adeguata:** Ogni coppia stato-azione (s, a) deve essere visitata un numero infinitamente grande di volte.
- **Decadimento di α :** Il learning rate deve diminuire progressivamente (es. $\alpha = \frac{1}{n(s, a)}$).

Il Q-Learning è intrinsecamente un algoritmo **Off-policy**: separa la policy usata per generare il comportamento (es. esplorazione epsilon-greedy) dalla policy che viene valutata e ottimizzata (puramente greedy, rappresentata dall'operatore $\max_{a'}$).

Lo pseudocodice dell'algoritmo Q-Learning è il seguente:

```
Initialize  $Q(s, a)$  arbitrarily for all  $s$  in  $S$ ,  $a$  in  $A(s)$ 
Initialize  $Q(\text{terminal-state}, \cdot) = 0$ 
Repeat (for each episode):
    Initialize  $s$ 
    Repeat (for each step of episode):
        Choose  $a$  from  $s$  using policy derived from  $Q$  (e.g., epsilon-greedy)
        Take action  $a$ , observe  $r, s'$ 
         $Q(s, a) \leftarrow Q(s, a) + \alpha * [r + \gamma * \max_{a'} Q(s', a') - Q(s, a)]$ 
         $s \leftarrow s'$ 
    until  $s$  is terminal
```

1.3 Esplorazione vs Sfruttamento

Un pilastro fondamentale del Reinforcement Learning è il delicato compromesso tra **esplorazione** (exploration) e **sfruttamento** (exploitation):

- **Esplorazione:** L'agente sacrifica il guadagno immediato per raccogliere informazioni che potrebbero portare a profitti molto più alti in futuro.
- **Sfruttamento:** L'agente sceglie le azioni che hanno dato buoni risultati in passato per ottenere ricompense immediate e sicure.

Per garantire la convergenza ad un risultato ottimo, è necessario esplorare tutte le coppie stato-azione con sufficiente frequenza nel lungo periodo. I principali metodi utilizzati nella pratica sono:

- **ϵ -greedy:** Scegliere l'azione in modo avido (*greedy*) per la maggior parte del tempo (con probabilità $1 - \epsilon$) e selezionare un'azione in modo casuale con probabilità ϵ .
- **Softmax (o Boltzmann):** tratta tutte le azioni esplorative allo stesso modo. Questo metodo assegna una probabilità di selezione a ciascuna azione ponderandola in base al suo valore atteso $Q(s, a)$.

La probabilità di scegliere l'azione a nello stato s è definita come:

$$p(a) = \frac{e^{Q(s,a)/T}}{\sum_{a'} e^{Q(s,a')/T}}$$

Il parametro T , noto come *temperatura*, agisce da modulatore cruciale per il grado di esplorazione:

- Quando T presenta un valore elevato, tutte le azioni hanno probabilità simili di essere estratte (**massima esplorazione**).
- Quando T presenta un valore basso, l'azione con il valore $Q(s,a)$ più alto ha una probabilità maggiore di essere scelta (**massimo sfruttamento**).

1.4 Spiegazione dei parametri

- **EPISODI:** Sequenze complete di interazioni agente-ambiente, dall'inizio fino allo stato terminale (goal) o al raggiungimento del limite massimo di passi (max_steps).
- α - **LEARNING RATE:** Determina il peso delle nuove informazioni rispetto a quelle passate. Regola la velocità di aggiornamento dei valori nella Q-table.
- γ - **DISCOUNT FACTOR:** Fattore che definisce l'importanza delle ricompense future; un valore vicino a 1 orienta l'agente verso una strategia di lungo periodo.
- ϵ - **EXPLORATION RATE:** Probabilità di scegliere un'azione casuale invece di quella ottimale stimata; fondamentale per esplorare l'ambiente.

2 AMBIENTE DOORKEY

2.1 Ambiente Doorkey

Per valutare le performance dell'agente è stato scelto l'ambiente DoorKey, della libreria MiniGrid. Questo ambiente presenta una chiave che l'agente deve raccogliere per sbloccare la porta e successivamente deve arrivare al quadrato verde che rappresenta il goal finale. Infatti la missione di questo ambiente è riassunta come: “usa la chiave per aprire la porta e poi raggiungi il goal”. Poiché la ricompensa viene fornita esclusivamente al completamento del task finale (ricompensa sparsa), l'ambiente rappresenta un'ottima sfida per testare le capacità di esplorazione dell'algoritmo.

Il reward di questo ambiente può essere solo di due tipi:

- $1 - 0.9 \cdot (\text{step_count} / \text{max_steps})$ se raggiunge il goal.
- 0 se fallisce.

L'episodio termina quando si verifica una delle seguenti condizioni:

- L'agente raggiunge con successo il *goal*.
- Viene superato il numero massimo di passi consentiti per un singolo episodio (definito dalla variabile *max_steps*).

2.1.1 Spazio delle Azioni

Le azioni possibili all'interno di questo ambiente sono 7, identificate da un numero:

1. **LEFT (0):** ruotare a sinistra.
2. **RIGHT (1):** ruotare a destra.
3. **FORWARD (2):** muoversi in avanti di una cella.
4. **PICKUP (3):** raccogliere un oggetto (la chiave).
5. **DROP (4):** rilasciare un oggetto.
6. **TOGGLE (5):** interagire con un oggetto (aprire la porta).
7. **DONE (6):** terminazione volontaria.

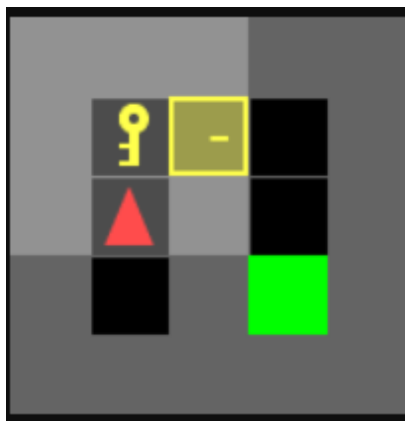


Figura 1: Esempio dell'ambiente DoorKey.

2.2 Scelta dell'algoritmo

Per l'addestramento dell'agente è stato impiegato l'algoritmo Q-Learning. Esso permette di stimare iterativamente la funzione di valore ottimo $Q(s, a)$ senza necessitare di una conoscenza a priori delle dinamiche di transizione dell'ambiente. Grazie all'utilizzo di una policy di esplorazione (come l' ϵ -greedy), l'agente è in grado di bilanciare la scoperta di nuove strategie con lo sfruttamento delle conoscenze già acquisite, un elemento cruciale per superare il problema delle ricompense sparse introdotto dalla meccanica della porta chiusa.

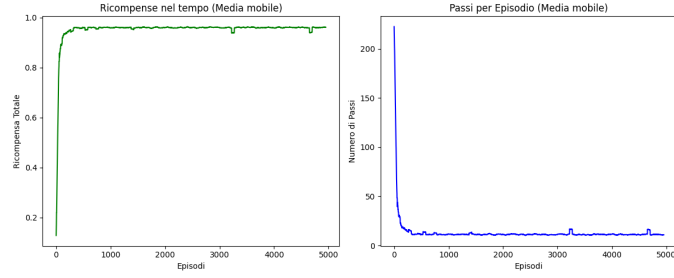
2.3 Setup dell'Addestramento

La fase di addestramento è stata strutturata in due diverse configurazioni sperimentali per valutare l'impatto dei parametri esplorativi e della durata massima degli episodi sulle performance di convergenza dell'agente. Entrambi i set di esperimenti sono stati condotti per un totale di 5000 episodi, mantenendo costanti il learning rate (α) e il fattore di sconto (γ). I test sono stati suddivisi come segue:

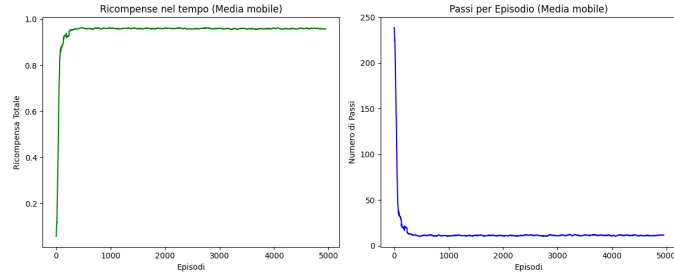
- **Configurazione 1 (Run 1-5):** Parametri impostati con $\alpha = 0.1$, $\gamma = 0.99$ ed un tasso di esplorazione iniziale $\epsilon = 0.9$. Il limite massimo di passi per episodio (`max_step`) è stato mantenuto al valore di default dell'ambiente, pari a 250.
- **Configurazione 2 (Run 6-10):** Parametri impostati con $\alpha = 0.1$, $\gamma = 0.99$. In questo caso, si è optato per una strategia di esplorazione iniziale più aggressiva, impostando $\epsilon = 1.0$. Inoltre, per concedere all'agente un orizzonte temporale più ampio per l'esplorazione casuale, il limite di passi è stato raddoppiato, portando il `max_step` a 500.

2.4 Risultati Sperimentali

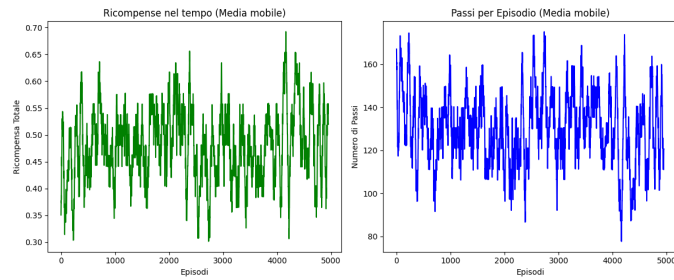
Nelle figure seguenti sono riportati i grafici relativi alle 10 esecuzioni (run) di addestramento. Le prime cinque misurazioni (**Figura 2**) fanno riferimento alla Configurazione 1, mentre le successive cinque (**Figura 3**) sono relative alla Configurazione 2.



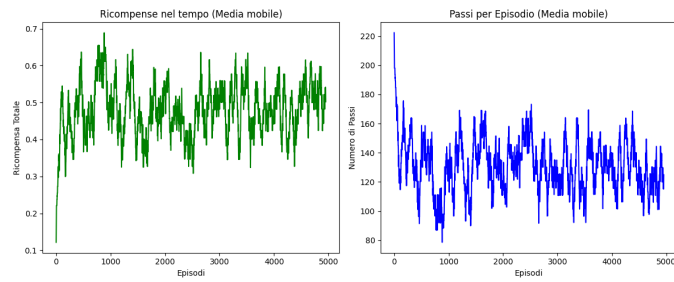
(a) Run 1



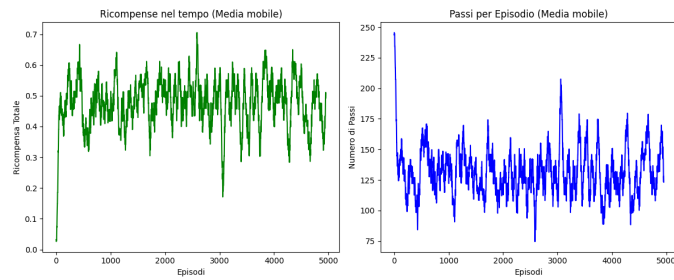
(b) Run 2



(c) Run 3

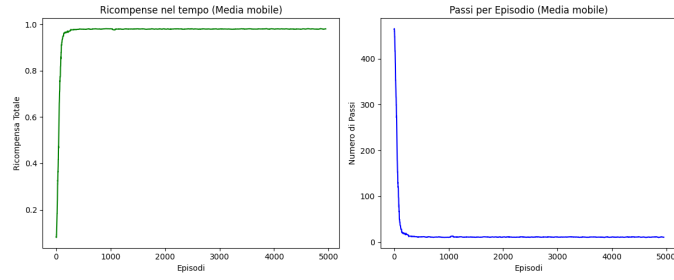


(d) Run 4

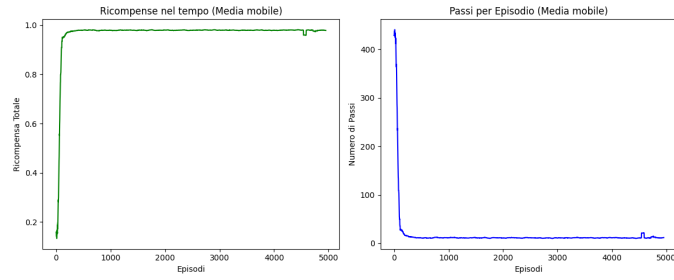


(e) Run 5

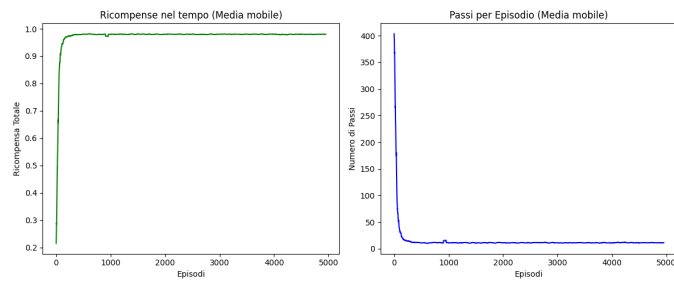
Figura 2: Risultati ottenuti con la Configurazione 1 ($\epsilon = 0.9$, $\text{max_step}=250$).



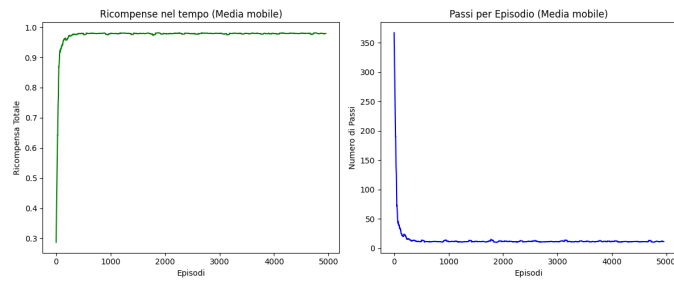
(a) Run 6



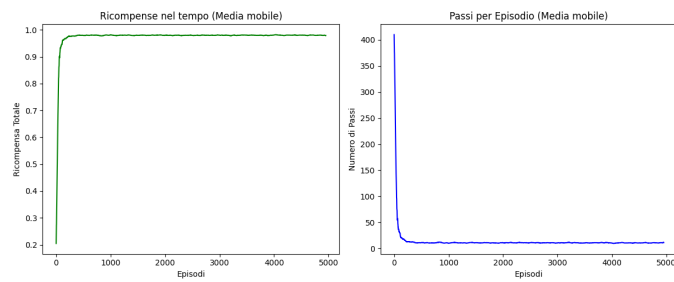
(b) Run 7



(c) Run 8



(d) Run 9



(e) Run 10

Figura 3: Risultati ottenuti con la Configurazione 2 ($\epsilon = 1.0$, $\text{max_step}=500$).

2.5 Analisi e Confronto delle Configurazioni

Osservando i dati raccolti, emerge chiaramente che la **Configurazione 2** riesce a ottenere risultati ottimali e, soprattutto, ampiamente riproducibili nel tempo, a differenza della Configurazione 1 che mostra una maggiore instabilità o una convergenza verso policy sub-ottimali.

Questo netto miglioramento delle performance è attribuibile a due fattori chiave:

1. **Maggiore orizzonte temporale:** L'incremento del parametro `max_step` da 250 a 500 fornisce all'agente il tempo materiale per completare con successo l'intera sequenza di azioni richieste dall'ambiente DoorKey (trovare la chiave, aprire la porta, raggiungere l'obiettivo) durante i primi episodi, quando i movimenti sono puramente stocastici.
2. **Esplorazione massimizzata:** L'utilizzo di un tasso di esplorazione iniziale più elevato ($\epsilon = 1.0$) assicura che l'agente esplori a fondo l'ambiente prima di iniziare ad affidarsi alla policy appresa (*exploitation*). In un ambiente a ricompensa sparsa come questo, esplorare maggiormente all'inizio previene la convergenza prematura e permette all'algoritmo di propagare correttamente i valori Q a ritroso dal goal fino agli stati iniziali.